

# UNIVERSITY COURSE REGISTRATION

UCS2265 – Fundamentals and Practice of Software Development

## A PROJECT REPORT

Submitted By

Krithik Sai S - 3122235001069

Hemanth Santhosh - 3122235001054

Dhananchezhiyan M - 3122235001041



Department of Computer Science and Engineering

Sri Sivasubramaniya Nadar College of Engineering  
(An Autonomous Institution, Affiliated to Anna University)  
Kalavakkam – 603110

June 2024

**Sri Sivasubramaniya Nadar College of Engineering**  
**(An Autonomous Institution, Affiliated to Anna University)**

**BONAFIDE CERTIFICATE**

Certified that this project report titled “**University Course Registration**” is the bonafide work of Krithik Sai (3122235001069), Hemanth Santhosh (3122235001054) and Dhananchezhiyan (3122235001041) who carried out the project work in the UCS2265 – Fundamentals and Practice of Software Development during the academic year 2023-24.

Internal Examiner

External Examiner

Date:

## TABLE OF CONTENTS

| Content                                                                      | Page Number |
|------------------------------------------------------------------------------|-------------|
| 1. Problem Statement                                                         | 04          |
| 2. Extended Exploration of Problem Statement                                 | 04          |
| 3. Data Flow Diagrams:                                                       |             |
| a. Level 0                                                                   | 05          |
| b. Level 1                                                                   | 05          |
| c. Level 2                                                                   | 06          |
| 4. System Architecture                                                       | 09          |
| 5. Modules and their Description                                             |             |
| a. Login Module                                                              | 10          |
| b. Choice Filling Module                                                     | 11          |
| c. Allotment Module                                                          | 12          |
| d. Add and Drop Course Module                                                | 13          |
| e. Waiting List Module                                                       | 15          |
| 6. Implementation:                                                           |             |
| a. Data Organization                                                         | 16          |
| b. Platform                                                                  | 17          |
| 7. Validation through different test cases                                   | 17          |
| 8. Limitations                                                               | 22          |
| 9. Observation through Social, Legal, Environmental,<br>Ethical perspectives | 23          |
| 10. Learning Outcomes                                                        | 24          |
| 11. References                                                               | 25          |

## PROBLEM STATEMENT

Develop a software system for taking care of course registration in a university. This system should

have the following features:

- Each student should log into the system to register for the chosen courses in each semester.
- Once the student logs in, a list of courses that are offered in that semester should be displayed along with the information about faculty members who are teaching each section of the course.
- Assume that each core course will have 3 faculties and each elective course will have two faculties.
- Students can choose the courses and their faculties in which they would like to register.
- Each student should register for a 5 core courses and 1 elective course for semester.
- At the end of the registration process, the list of successfully allotted courses along with the name of the teacher for each course should be displayed. In addition, the list of waitlisted courses along with waitlist number should also be displayed.

## EXTENDED EXPLORATION OF PROBLEM STATEMENT:

### Executive summary :

The course registration software system can streamline and automate the registration process for students, addressing challenges of the current manual system such as errors, inefficiencies, and limited data for decision-making.

The software should allow students to log in, choose courses with a minimum of four and a maximum of six per semester, and display the list of successfully allotted courses with the specific available faculty 's name. A waitlist should also be maintained.

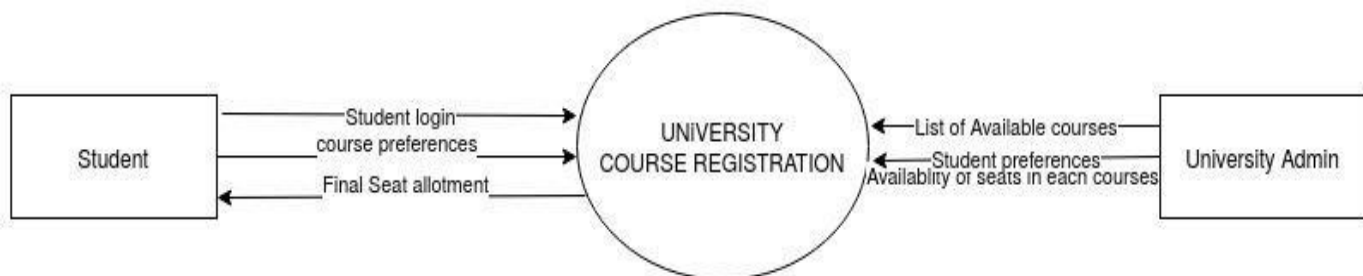
Constraints such as prerequisites, fixed faculty for each course, and a waitlist (10% of the class strength) should be implemented. Students should be able to add or drop courses within the first two weeks after the semester starts.

- Admin has the functionality to check course availability, waitlist status, and other important announcements via the admin login module.
- Sorting the students in the waiting list based on the additional criteria like additional courses completed, and other extra curriculars by giving appropriate points for each student.

### **Analysis using data flow diagram:**

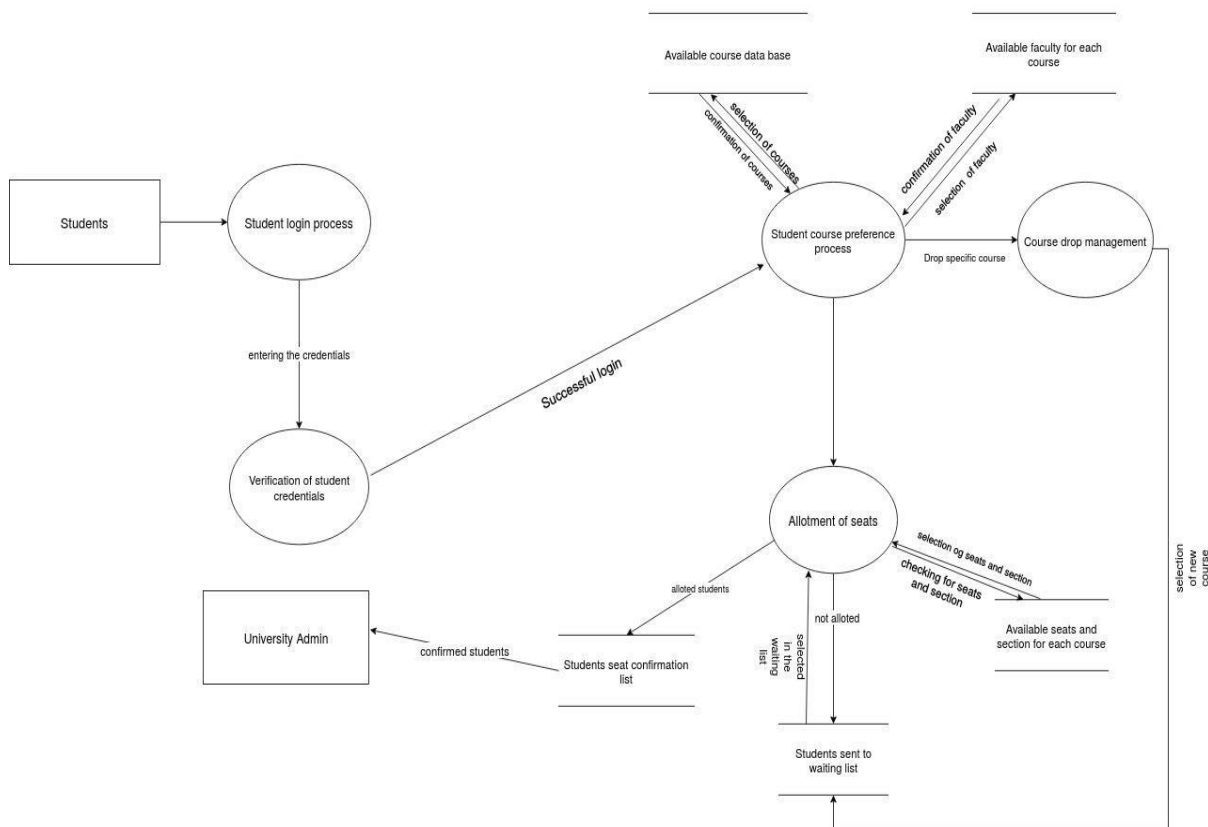
(Level 0):

This level 0 DFD shows the basic structure of the logic behind the program. Here the 2 entities using the program will be the student and the university administrator. Students will first login and then provide their preferences to the program. While the university admin will provide the data for the available courses and available seats in each course. The program will respectively allocate the seats to each student based on the prerequisites and this is intimated to the student.



(Level 1):

The first step is for the students to login to the system. If the login credentials are correct then the student will move on the next step which is choosing the course preferences. The database containing the available courses and faculty details is checked. The next step is each student's preference is stored and passed onto another module which allots the students into their respective seats. This module will create a file containing the students who have confirmed seats and another one putting students into a waiting list. The students who have confirmed seats are also given the option to drop specific courses and add new course if they choose to do so.

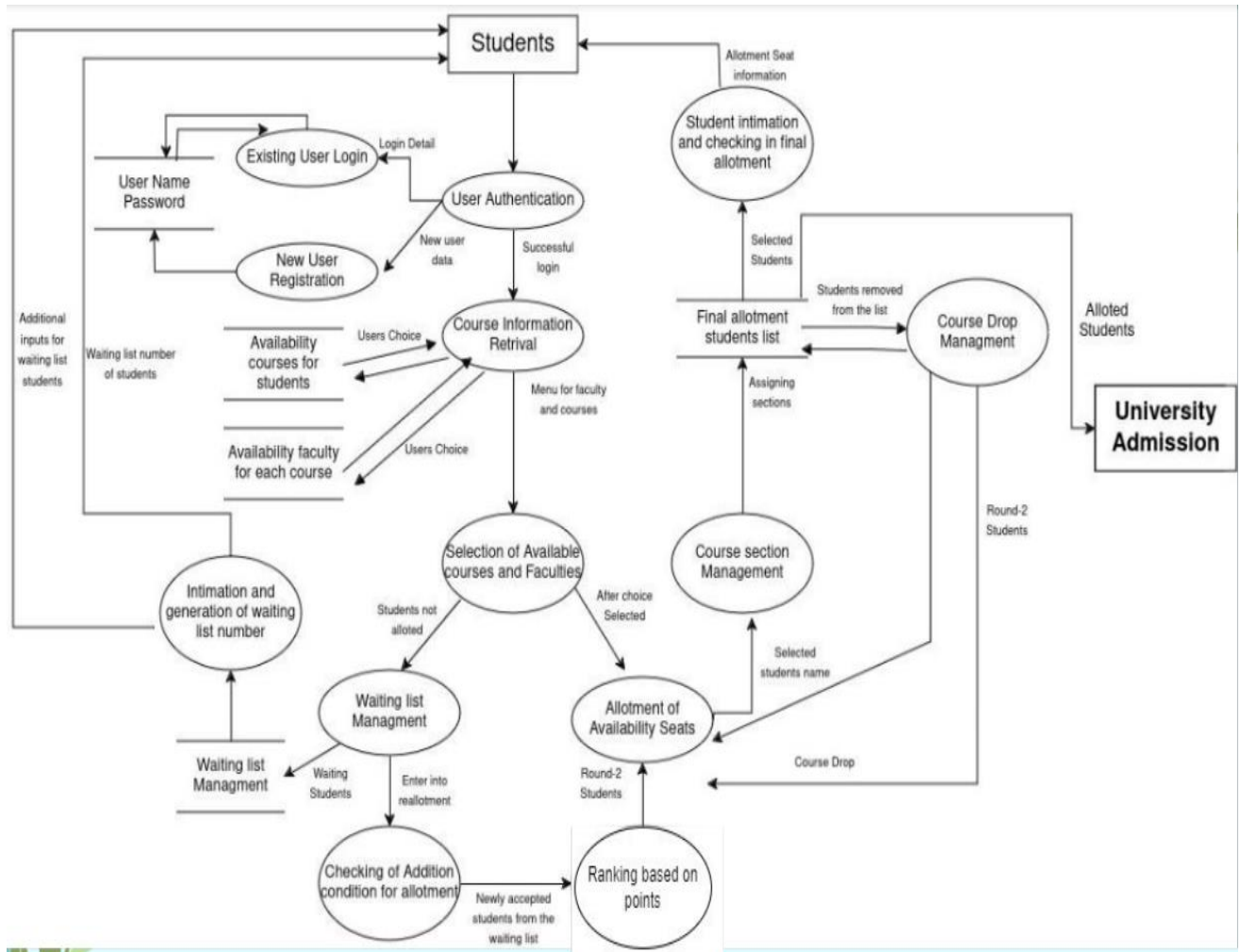


(Level 2):

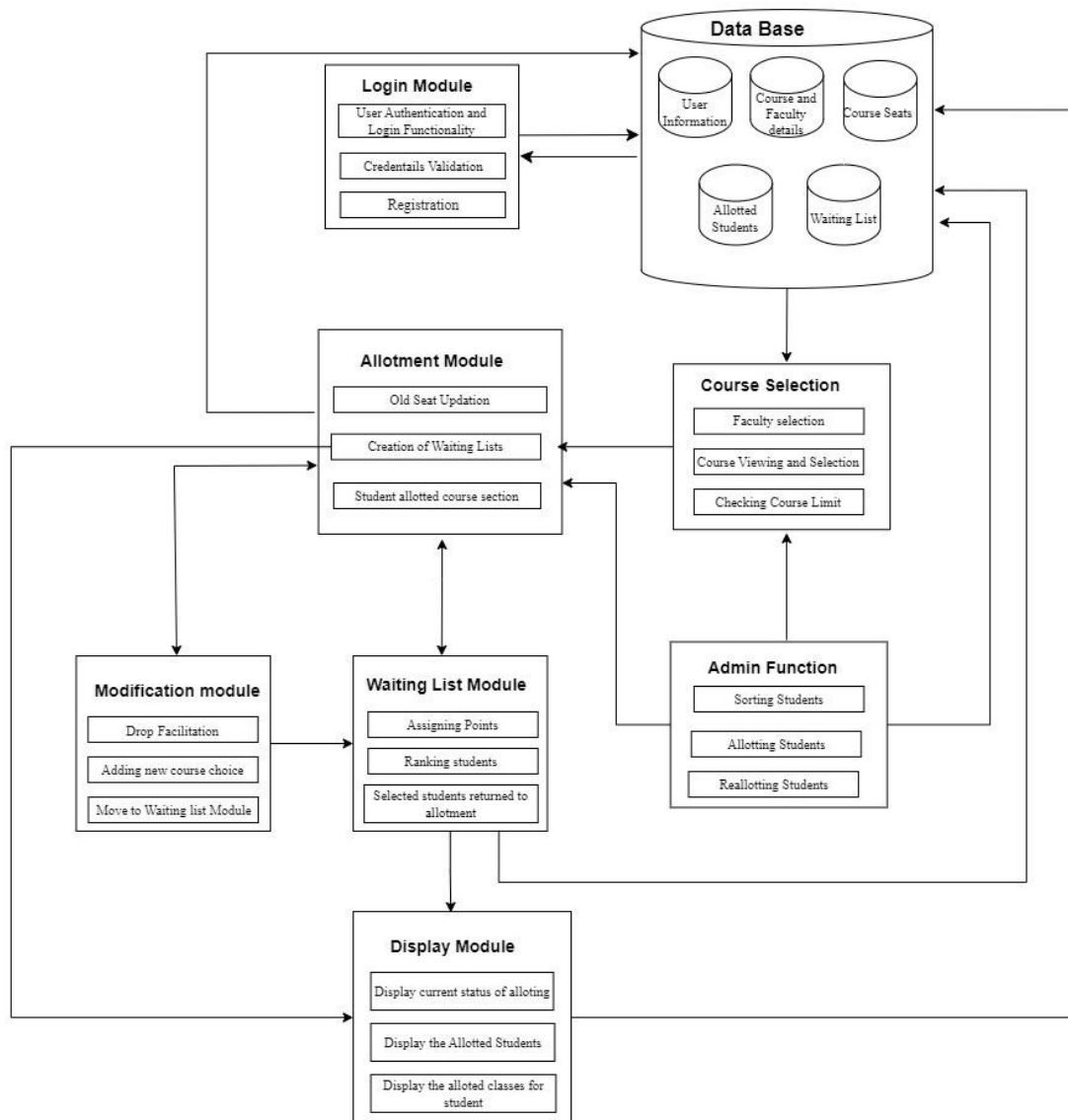
The students first start with the user authentication module, this module has two sub processes either to register a new user or to login by an existing user. The username and password for each student is stored in a database. On successful login the details regarding the available courses and the faculty teaching it will be displayed. This information is also retrieved from 2 separate databases. Here the choices of each student is recorded and then this data is passed onto the next module. In this module the students are allotted into specific courses based on the specified prerequisites. Then it goes to the next module where ranking is done based on the previous semester CGPA marks.

These students are then allotted into classes of definite number of students. The final allotted students are added into a database, if a student chooses to drop a course then they are led into a drop course and can add a new course and the module will modify the original database.

The students who are not allotted in the classes are managed by the waiting list module which creates a separate database containing these students. The waitlisted students are further ranked basis of external factors by another module and finally students may get allotted or not based on marks. The final status of each student is intimated to students and the final allotted student database is in provided to the administrator.



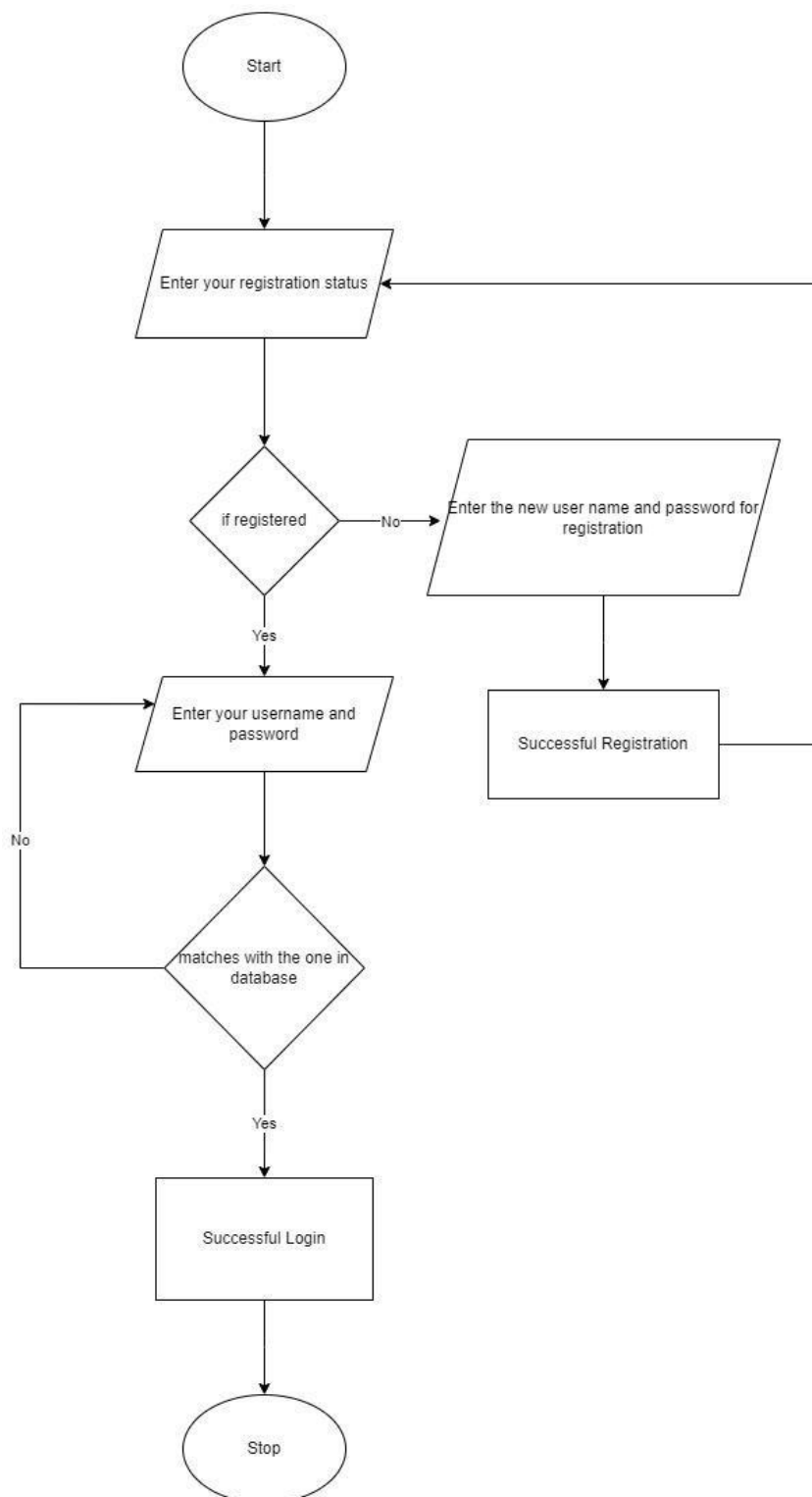
**Detailed Design:**  
(Overall Architectural Diagram):



**Modules and their description:**



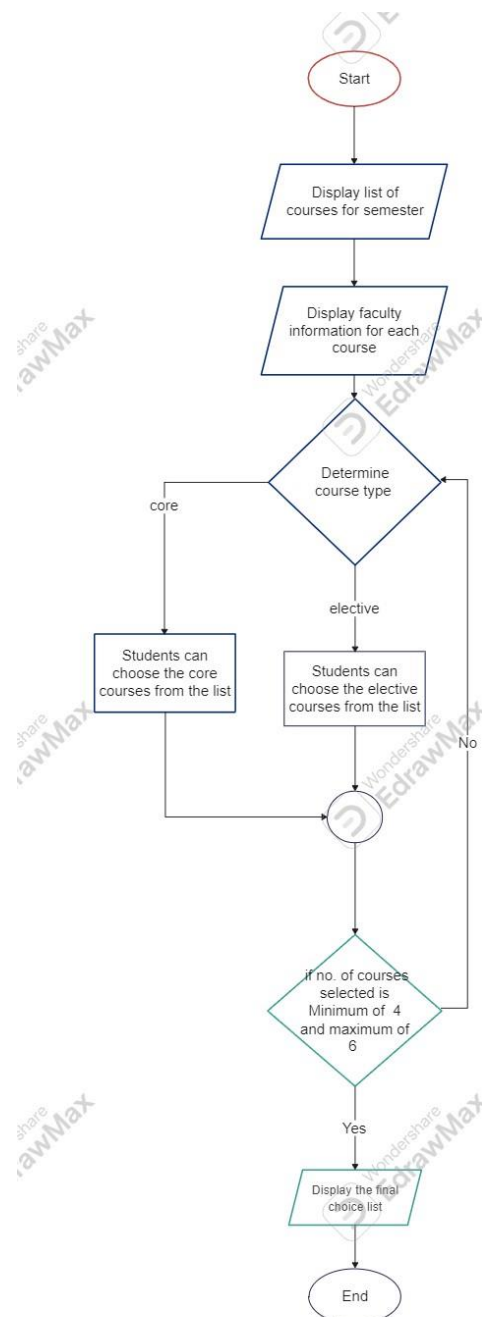
## 1.Login module:



The Structure Chart that begins with the Login page. First the user have to enter the details to get registered. If already registered, then enter the user name and password . If it gets matched with the database, then user successfully logs in. Otherwise, reenter the user name and password.

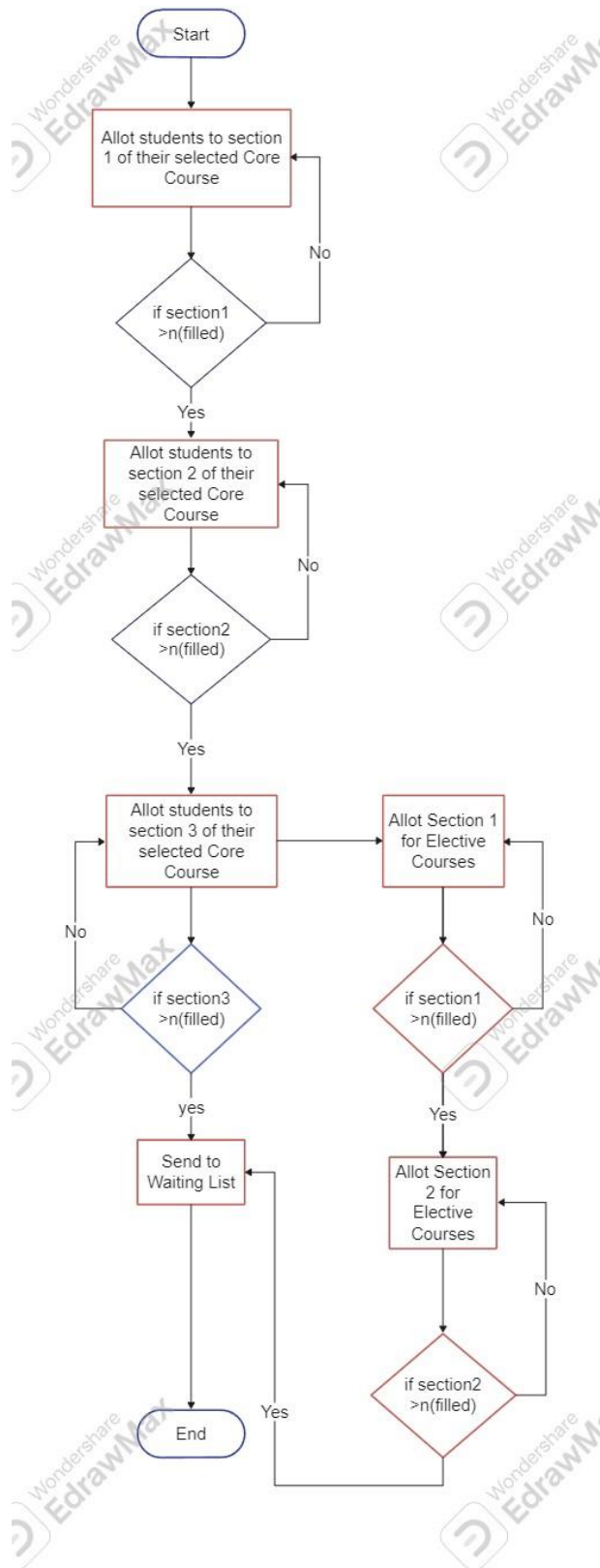
If not already registered, goes to the page where user has to enter the new user name and password. If the registration is successful, then it will enter into the registration status.

## 2.Choice filling module:



The User can select the specific course and the number of courses have to be 6 of maximum (5 core courses and 1 elective course).

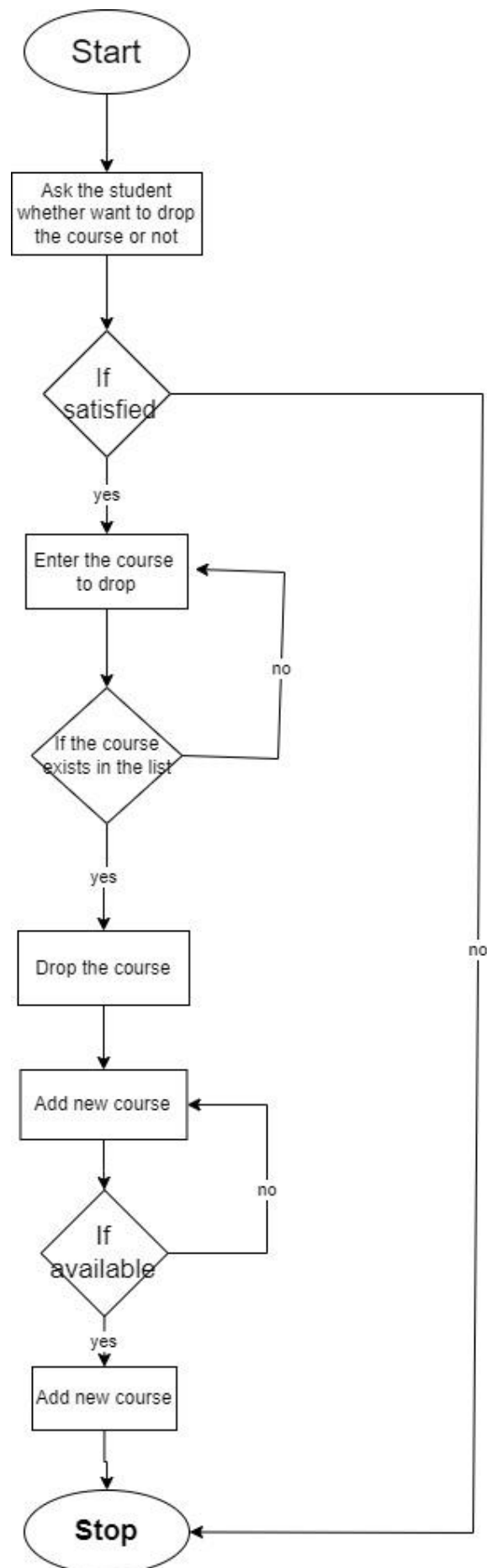
### 3.Allottment module:



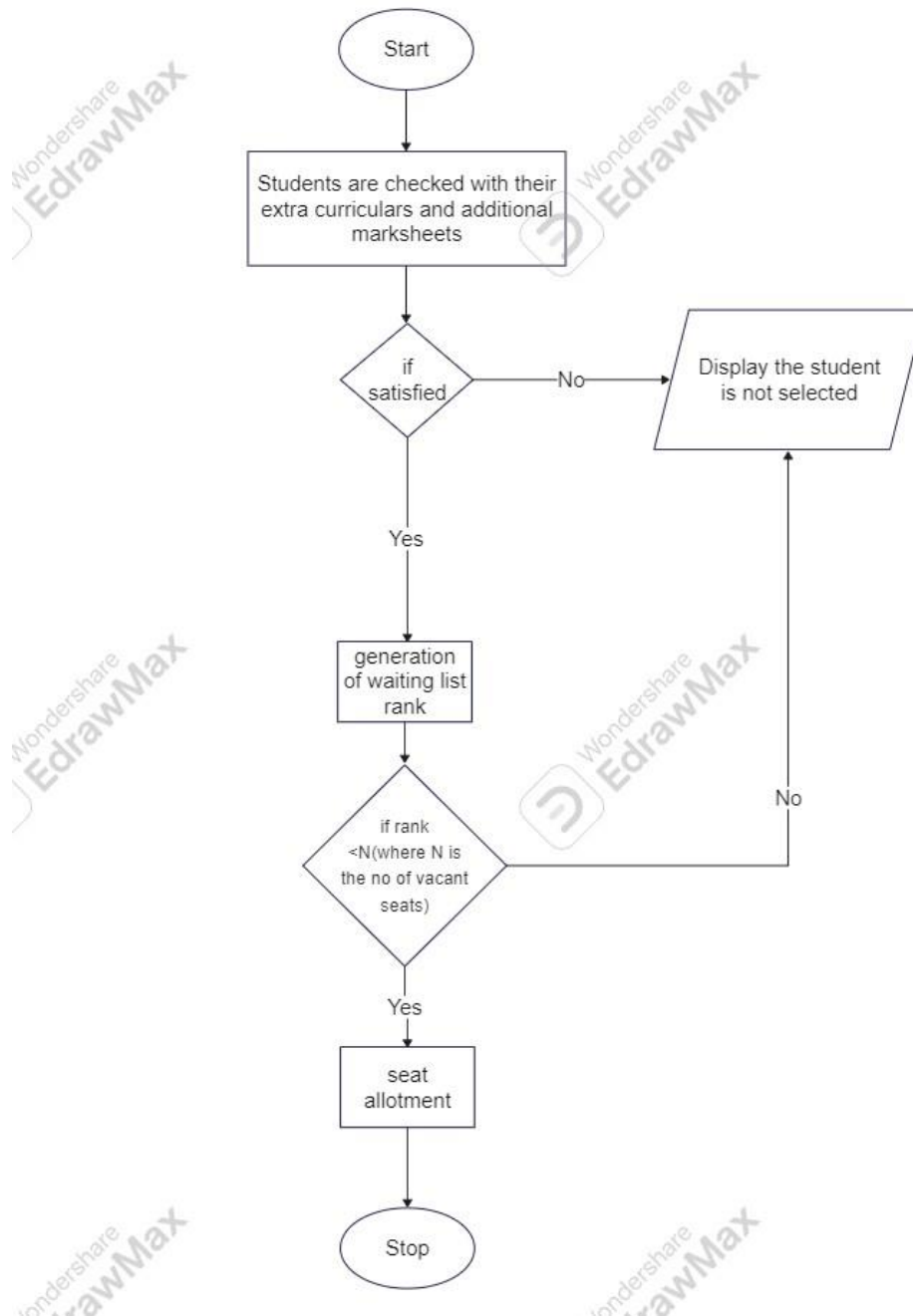
Allotting the students to specific classes is done in this module. The students of each course are taken into this module and ranking is done based on the CGPA scored in the previous semester. When each class is fully filled in strength the student is allotted to the next class. The students who are not having place in each class will move to the waiting list. The course and the selected faculty are the constraints for adding the students to classes.

#### 4. Add and Drop Course module:

- **Course Selection:** Students can browse the list of available courses, including core and elective options. Each course listing includes details such as course name, instructor, available sections, and remaining seats.
- **Seat Availability:** The system checks for available seats in the selected course and section. If seats are available, the course is added to the student's schedule.
- **Waiting List:** If the selected course is full, the student can opt to be placed on a waiting list. The system ranks students on the waiting list based on predefined criteria, such as academic performance, hackathon wins, and reservation status.
- **Course Listing:** Students can view their current course enrollments, including core and elective courses. Each course entry displays pertinent details like course name, instructor, and schedule.
- **Drop Confirmation:** To drop a course, the student selects the course from their schedule and confirms the drop action. The system ensures that the student does not fall below the minimum required number of courses for the semester.
- **Seat Reallocation:** When a course is dropped, the system updates the seat availability for that course. If there is a waiting list for the dropped course, the system reallocates the seat to the highest-ranked student on the waiting list.



### 5.Waiting list module:



For this module , the students in the waiting list are given a chance to enter the marks obtained in other examinations, this will act as a way to prioritise them in the allotment process . Based on the marks they are given rank and get allotted in the respective classes.

## Implementation: Data Organization:

The data is organized using `typedef` structures to encapsulate related information, ensuring clarity and ease of access. Here's a breakdown of how the data is organized and the rationale behind selecting the C language:

### 1. Constants and ANSI Escape Codes:

- The `#define` preprocessor directives create constants for various maximum values, like `MAX\_CHOICES` and `MAX\_STUDENTS`.
- ANSI escape codes are defined to facilitate text formatting and coloring in the terminal, improving the user interface.

### 2. Data Structures:

- Student: This structure holds information about individual students, including their username, password, semester, GPA, and their selected courses along with choices. The nested structure within `Student` for courses ensures that each student's course information is tightly coupled with their profile.
- Core: This structure represents core courses with a name and seats available in three sections.
- waitlist Stud: This structure is used to store information about students on the waitlist, including their username and CGPA.
- User: This comprehensive structure contains detailed information about a user, such as their username, password, semester, CGPA, core courses, core faculty preferences, elective courses, and elective faculty preferences.
- Allot: This structure represents the allotment preferences for a course, holding the course name and faculty preferences.
- Course: A simple structure for storing the course name.
- Allocation: This structure is used to keep track of course allocations, including the username, course, and section.
- AllottedStudent: This structure captures the courses and sections allotted to a student.
- Course\_seat: This structure represents a course and the number of available seats in each of its three sections.
- Wait\_points: This structure stores information about students on the waitlist, including their GPA and choice preferences along with points for prioritization.

### Rationale for Using C:

- Performance: C is a compiled language that provides high performance and efficient memory management, which is crucial for handling large datasets (such as hundreds of students and their course allocations).
- Control: C allows fine-grained control over system resources, enabling the creation of optimized and efficient code for complex operations like course registration and allocation.
- Portability: C is a portable language, meaning the code can be compiled and run on various hardware and operating systems with minimal modification.

- **Simplicity and Direct Access:** C provides direct access to memory and system resources, which is beneficial for implementing detailed and customized functionality required in a course registration system.
- **Legacy and Integration:** Many existing systems and libraries are written in C, making it easier to integrate new systems with legacy codebases or utilize existing libraries.

Overall, the data organization using structured types in C provides a clear, efficient, and manageable way to handle complex data relationships, which is essential for a robust university course registration system.

Platform Used

### Platform:

For the implementation of the university course registration system, we utilized **Code::Blocks**, an open-source, cross-platform Integrated Development Environment (IDE). Code::Blocks was chosen due to its robust features, ease of use, and support for multiple compilers, making it suitable for our C programming needs.

### Validation through detailed test cases for various Scenarios:

#### Test Case 1: User Authentication

**Scenario:** Successful Login

- **Input:**
  - Username: "student123"
  - Password: "password123"
- **Expected Output:**
  - Message: "Login successful. Welcome, student123!"
  - Redirect to course selection page.
- **Actual Output:**

```
ENTER YOUR USERNAME: krithi
ENTER YOUR PASSWORD: krithishetty
You are logged in successfully
```



### Scenario: Unsuccessful Login (Incorrect Password)

- **Input:**
  - Username: "student123"
  - Password: "wrongpassword"
- **Expected Output:**
  - Message: "Invalid username or password. Please try again."
- **Actual Output:**

```

Welcome

=====
                                UNIVERSITY COURSE SELECTION SYSTEM

MAIN MENU

1. REGISTRATION
2. LOGIN
3. ADMIN LOGIN
4. EXIT

Enter your choice: 2
ENTER YOUR USERNAME: krithi
ENTER YOUR PASSWORD: krithin

Username or password is wrong.
Please enter a valid username or password.

You have 2 attempts remaining.

```

### Test Case 2: New User Registration

#### Scenario: Successful Registration

- **Input:**
  - New Username: "newstudent"
  - Password: "newpassword"
- **Expected Output:**
  - Message: "Registration successful. Please log in."
- **Actual Output**

```

REGISTER PAGE

The username should have at least 5 characters and at most 10 characters.

Enter the username:hibal23

Password should contain at least 8 characters.

Enter the password: hiba1234

Username and Password are Successfully Registered

```

### Test Case 3: Course Selection

#### Scenario: Successful Course Selection

- **Input:**
  - Core Courses: ["Course1", "Course2", "Course3", "Course4", "Course5"]
  - Elective Course: "Elective1"
- **Expected Output:**
  - Message: "Courses successfully registered."
- **Actual Output:**

```

Choose the elective course:
[1] Psychology
[2] Film Appreciation
Enter the course number:2

Subject: Film Appreciation
Info of faculty:
  Film Appreciation
  Faculty 1: Dr. Michael Johnson
    12 years of experience
    Alumni rating: 4.9
  Faculty 2: Prof. Emily Brown
    9 years of experience
    Alumni rating: 4.7

Enter your 1 preference of faculty for Elective (1 or 2): 1
Enter your 2 preference of faculty for Elective (1 or 2): 2

YOUR CHOICES HAVE BEEN SUCCESSFULLY REGISTERED

```

**Scenario:** Selection of Wrong Course

- **Input:**
  - 9
- **Expected Output:**
  - Message:Invalid course number.
- **Actual Output:**

```

Available Core courses:
[1] Design Thinking
[2] Machine learning
[3] Data Science
[4] Applied Mathematics
[5] Robotics
[6] Data Architecture
[7] Theoretical Physics

Students are required to choose 5 core subjects this semester:

Enter the course [1] number: 9
Invalid course number.

```

#### Test Case 4: Course Allotment

**Scenario:** Course Allotment Based on CGPA

- **Input:**
  - CGPA: 8.5
  - Course Preferences: ["Course1", "Course2", "Course3", "Course4", "Course5"]
- **Expected Output:**
  - Message: "Courses allotted based on CGPA."
  - Display list of allotted courses and faculty.

- **Actual Output:**

|            |             |   |      |            |   |   |   |            |   |   |   |            |   |   |   |            |   |   |   |            |   |   |
|------------|-------------|---|------|------------|---|---|---|------------|---|---|---|------------|---|---|---|------------|---|---|---|------------|---|---|
| charlie    | charlie123  | 2 | 9.9  | Theoretica | 1 | 2 | 3 | Data Scien | 1 | 2 | 3 | Machine le | 1 | 2 | 3 | Applied Mi | 1 | 2 | 3 | Design Thi | 1 | 2 |
| aaftab     | aaftab123   | 2 | 9.7  | Design Thi | 1 | 2 | 3 | Machine le | 1 | 2 | 3 | Applied Mi | 1 | 2 | 3 | Robotics   | 1 | 2 | 3 | Data Archi | 1 | 2 |
| reacher    | jack1234    | 2 | 9.02 | Data Scien | 3 | 2 | 1 | Robotics   | 3 | 2 | 1 | Data Archi | 3 | 2 | 1 | Theoretica | 1 | 2 | 3 | Design Thi | 2 | 3 |
| sudharsan  | rupasound   | 2 | 9    | Design Thi | 1 | 2 | 3 | Machine le | 1 | 2 | 3 | Data Archi | 1 | 2 | 3 | Robotics   | 1 | 2 | 3 | Theoretica | 1 | 2 |
| Maniiiiii  | mani1234    | 2 | 9    | Design Thi | 1 | 2 | 3 | Machine le | 1 | 2 | 3 | Applied Mi | 1 | 2 | 3 | Data Scien | 1 | 2 | 3 | Data Archi | 1 | 2 |
| HARISH     | harish123   | 2 | 9    | Design Thi | 1 | 2 | 3 | Applied Mi | 1 | 3 | 2 | Machine le | 1 | 3 | 2 | Robotics   | 1 | 3 | 2 | Data Archi | 3 | 2 |
| krithi     | krithishett | 2 | 8.4  | Design Thi | 1 | 2 | 3 | Applied Mi | 1 | 2 | 3 | Machine le | 1 | 2 | 3 | Data Archi | 1 | 2 | 3 | Robotics   | 1 | 2 |
| dhinesh    | mithra123   | 2 | 7.3  | Design Thi | 1 | 2 | 3 | Applied Mi | 1 | 2 | 3 | Robotics   | 1 | 2 | 3 | Data Scien | 1 | 2 | 3 | Data Archi | 1 | 2 |
| dhinesh    | mithra123   | 2 | 7.3  | Design Thi | 1 | 2 | 3 | Machine le | 1 | 2 | 3 | Data Scien | 1 | 2 | 3 | Robotics   | 1 | 2 | 3 | Data Archi | 1 | 2 |
| nithya     | nithy123    | 2 | 7.01 | Applied Mi | 1 | 2 | 3 | Data Scien | 2 | 3 | 1 | Machine le | 1 | 2 | 3 | Data Archi | 2 | 3 | 1 | Design Thi | 1 | 2 |
| justustonk | jesus123    | 2 | 6.85 | Theoretica | 1 | 2 | 3 | Applied Mi | 3 | 2 | 1 | Design Thi | 2 | 3 | 1 | Machine le | 3 | 1 | 2 | Robotics   | 2 | 3 |
| Vanitha    | Vanitha12   | 2 | 4.8  | Data Archi | 2 | 1 | 3 | Machine le | 1 | 2 | 3 | Robotics   | 1 | 2 | 3 | Design Thi | 2 | 1 | 3 | Data Scien | 1 | 2 |

The students have been successfully allotted...

|  |            |              |   |              |              |            |              |            |            |              |              |            |            |              |            |            |            |   |
|--|------------|--------------|---|--------------|--------------|------------|--------------|------------|------------|--------------|--------------|------------|------------|--------------|------------|------------|------------|---|
|  | charlie    | Theoretica   | 3 | Data Scien W | Machine le   | 3          | Applied Ma   | 3          | Design Thi | 3            | Psychology   | 1          |            |              |            |            |            |   |
|  | aaftab     | Design Thi   | 3 | Machine le   | 3            | Applied Ma | 3            | Robotics   | 3          | Data Archi   | 3            | Film Appre | 2          |              |            |            |            |   |
|  | reacher    | Data Scien W |   | Robotics     |              | 3          | Data Archi   |            | 3          | Theoretica   | 3            | Design Thi | 3          | Film Appre   | 2          |            |            |   |
|  | sudharsan  | Design Thi   |   | 3            | Machine le   | 3          | Data Archi   |            | 3          | Robotics     |              | 3          | Theoretica | 3            | Psychology | 1          |            |   |
|  | Maniiiiii  | Design Thi   |   | 3            | Machine le   | 3          | Applied Ma   |            | 3          | Data Scien W |              | Data Archi |            | 3            | Film Appre | 2          |            |   |
|  | HARISH     | Design Thi   |   | 3            | Applied Ma   | 3          | Machine le   |            | 3          | Robotics     |              | 3          | Data Archi |              | 3          | Psychology | 1          |   |
|  | krithi     | Design Thi   |   | 3            | Applied Ma   | 3          | Machine le   |            | 3          | Data Archi   |              | 3          | Robotics   |              | 3          | Psychology | 1          |   |
|  | dhinesh    | Design Thi   |   | 3            | Applied Ma   |            | 3            | Robotics   |            | 3            | Data Scien W |            | Data Archi |              | 3          | Psychology | 1          |   |
|  | dhinesh    | Design Thi   |   | 3            | Machine le   | 3          | Data Scien W |            | Robotics   |              | 3            | Data Archi |            | 3            | Psychology | 1          |            |   |
|  | nithya     | Applied Ma   |   | 3            | Data Scien W |            | Machine le   |            | 3          | Data Archi   |              | 3          | Design Thi |              | 3          | Film Appre | 2          |   |
|  | justustonk | Theoretica   |   | 3            | Applied Ma   |            | 3            | Design Thi |            | 3            | Machine le   |            | 3          | Robotics     |            | 3          | Psychology | 1 |
|  | Vanitha    | Data Archi   |   | 3            | Machine le   |            | 3            | Robotics   |            | 3            | Design Thi   |            | 3          | Data Scien W |            | Film Appre | 2          |   |

### Scenario: Full Class and Waitlist Allotment

- **Input:**
  - CGPA: 7.0
  - Course Preferences: ["Course1", "Course2", "Course3", "Course4", "Course5"]
- **Expected Output:**
  - Display list of allotted courses, faculty, and waitlisted courses with waitlist number.
- **Actual Output:**

You have already selected your courses.

Allotted Courses for Vanitha:

Course: Data Architecture, Section: 3

Course: Machine learning, Section: 3

Course: Robotics, Section: 3

Course: Design Thinking, Section: 3

Course: Data Science, Section: W

Course: Film Appreciation, Section: 2

### Test Case 5: Drop and Add Courses

**Scenario:** Successful Drop and Add

- **Input:**
  - Drop Course: "Course3"
  - Add Course: "Course6"
- **Expected Output:**
  - Message: "Course dropped and new course added successfully."
  - Display updated list of registered courses.
- **Actual Output:**

```

1.Do you want to modify your course selection?

2. To enter details for waitlist ranking.

3.Exit

        Enter your choice: 1
Your current core courses are:
1. Data Architecture (Section 3)
2. Machine learning (Section 3)
3. Robotics (Section 3)
4. Design Thinking (Section 3)
5. Design Thinking (Section 3)
Enter the number of the course you want to drop: 1
Available core courses to add:
3. Data Science
4. Applied Mathematics
6. Data Architecture
7. Theoretical Physics
Enter the number of the course you want to add: 7
Your changes have been recorded....
LOGIN AGAIN TO CHECK RESULTS

```

**Test Case 6: Waitlist Management****Scenario:** Waitlist Priority Based on Additional Points

- **Input:**
  - Waitlisted Course: "Course1"
  - Additional Points: 20
- **Expected Output:**

Display updated waitlist position.

- **Actual Output:**

```

C:\Users\Kishu\Desktop\review> Data
Vanitha,4.80,1,2,3,14

```

```

You have already selected your courses.
Allotted Courses for Vanitha:
Course: Data Architecture, Section: 3
Course: Machine learning, Section: 3
Course: Robotics, Section: 3
Course: Design Thinking, Section: 3
Course: Data Science, Section: W
Course: Film Appreciation, Section: 2

1.Do you want to modify your course selection?
2. To enter details for waitlist ranking.
3.Exit

Enter your choice: 2

Have you won any hackathons (yes/no)? yes
Are you 7.5% reservation (yes/no)? no
Have you participated in any district level sports (yes/no)? yes
Have you done any research projects (yes/no)? no
Have you done any extra courses in online platforms like Coursera or Udemey (yes/no)? yes

```

## Limitations:

### 1. Lack of CGPA Verification:

- The system does not have a mechanism to verify if the students actually possess the CGPA they claim.
- This limitation can lead to misuse by students who might enter a higher CGPA to gain an advantage in course selection or allocation.

### 2. Course Completion Verification:

- The system does not verify if students have actually completed prerequisite courses before allowing them to register for advanced courses.
- This can result in students enrolling in courses without having the necessary foundational knowledge, potentially affecting their academic performance.

### 3. Single Semester Operation:

- The system is designed to operate for only one semester at a time, limiting its ability to manage long-term academic planning and multi-semester course dependencies.
- This restriction can inconvenience students and administrators who need to plan and allocate resources for multiple semesters.

The university course registration system is currently limited in several key areas. It lacks a robust mechanism for verifying students' CGPA and course completion status, allowing students to potentially misuse the system by entering inaccurate academic information. Additionally, the system is designed to manage course registrations for only one semester at a time, which restricts its ability to support long-term academic planning and multi-semester course dependencies. These limitations highlight the

need for enhanced verification processes and a more comprehensive multi-semester operational capability to ensure the system's integrity and effectiveness.

### **Observation from the social, Legal, environmental and Ethical perspectives:**

#### **Social Perspective**

##### **Accessibility and Inclusion:**

- The system must be accessible to all students, including those with disabilities, by supporting screen readers and keyboard navigation.
- Offering multilingual support can help non-native speakers use the system effectively.
- Addressing the digital divide by providing alternative registration methods for students without reliable internet access or necessary devices.

##### **User Experience:**

- The system should be intuitive and user-friendly, minimizing the learning curve for new users.
- Providing adequate help and support services for students encountering difficulties.

##### **Impact on Student Life:**

- A well-functioning system reduces stress and anxiety among students by offering a smooth registration process.
- Implementing fair algorithms ensures equitable opportunities for all students in course allocation.

#### **Legal Perspective**

##### **Data Privacy and Security:**

- The system must comply with data protection laws such as GDPR or FERPA.
- Implementing robust data security measures, including encryption and secure login protocols.

##### **Intellectual Property:**

- Ensuring all software components are properly licensed.
- Clearly defining the ownership of content and data within the system, protecting students' personal information.

##### **Compliance with Educational Standards:**

- Aligning the system with accreditation requirements and relevant educational standards.

#### **Environmental Perspective**

##### **Sustainable Practices:**

- Optimizing the system for energy efficiency to reduce its carbon footprint.
- Considering cloud computing services that use renewable energy.

#### **Resource Utilization:**

- Digitizing the registration process reduces the need for paper forms and documents.
- Implementing policies for the proper disposal and recycling of electronic devices.

#### **Ethical Perspective**

##### **Fairness and Non-Discrimination:**

- Ensuring algorithms for course allocation are transparent and unbiased.
- Designing the system to provide equitable access for all students.

##### **Data Ethics:**

- Obtaining informed consent from students before collecting their personal data.
- Collecting only necessary data to minimize the risk of misuse.

##### **Professional Responsibility:**

- Developers and administrators should be accountable for the system's performance and ethical implications.
- Regularly reviewing and updating the system to address ethical concerns and adhere to best practices in technology and education.

## **Learning Outcomes from the University Course Registration System Project:**

Working as a team of three on the university course registration system project, we gained valuable experience and insights. Here are the key learning outcomes based on our reflections:

### **1. Collaboration and Teamwork**

- **Effective Communication:** We learned the importance of clear and regular communication within the team. By conducting regular meetings and discussions, we ensured everyone was on the same page and could contribute effectively.
- **Task Distribution:** We developed skills in dividing tasks based on individual strengths and areas of expertise. This helped in optimizing our workflow and ensuring that each team member could focus on their assigned responsibilities.
- **Conflict Resolution:** We encountered and resolved differences in opinions and approaches through constructive discussions, which taught us how to handle conflicts and find common ground.

### **2. Technical Skills**

- Programming Proficiency: Our proficiency in C programming improved significantly. We enhanced our understanding of concepts like file handling, dynamic memory allocation, and data structures.
- Using Code::Blocks IDE: We became adept at using Code::Blocks for developing, debugging, and testing our code. This experience made us more comfortable with IDE tools and features, including project management and version control.
- Algorithm Design: We honed our skills in designing algorithms for various features of the system, such as course allocation, waiting list management, and point calculation based on criteria.

### 3. Project Management

- Planning and Execution: We learned how to plan and execute a project from start to finish. This included setting milestones, adhering to deadlines, and monitoring progress.
- Documentation: We understood the importance of maintaining comprehensive documentation. This included writing clear and detailed reports, comments within the code, and user manuals.
- Testing and Debugging: We developed systematic approaches to testing and debugging our code. By creating test cases and scenarios, we ensured the system's robustness and reliability.

### 4. Problem-Solving and Critical Thinking

- Handling Complex Requirements: We learned how to break down complex requirements into manageable tasks. This helped us tackle challenging aspects of the project, such as integrating various system components and managing data consistency.
- Adapting to Changes: We became flexible in adapting to changes in requirements and scope. This included making adjustments based on feedback and unexpected challenges during implementation.
- Innovative Solutions: We developed creative solutions to problems, such as designing the waiting list ranking system and implementing effective course selection mechanisms.

### 5. Reflection and Continuous Improvement

- Self-Assessment: Reflecting on our individual and team performance helped us identify areas for improvement. We recognized our strengths and weaknesses, which will guide our future projects.
- Learning from Mistakes: We acknowledged and learned from the mistakes we made during the project. This experience taught us the value of iterative improvement and continuous learning.

### Conclusion

Overall, the experience of working on the university course registration system project as a team was highly educational and enriching. It not only improved our technical skills but also taught us valuable lessons in collaboration, project management, and problem-solving. These learning outcomes have equipped us with the knowledge and skills necessary for future projects and professional endeavours.



## REFERENCES:

### Tamil Nadu Engineering Admissions (TNEA) Official Website:

**Purpose:** Served as a real-world example and reference for admission and course registration processes.

**Contributions:**

- Provided information on how course allocations are handled, including reservation policies, merit lists, and counseling procedures.
- Offered insights into multi-step admission processes and handling large volumes of student data.
- <https://www.tneaonline.org/>

### Geeks for Geeks:

- **Purpose:** Used as a technical reference for programming concepts, C language features, and implementation strategies.
- **Contributions:**
  - Provided detailed explanations and examples of file handling, data structures, and algorithms in C.
  - Offered code snippets and tutorials for implementing specific features such as sorting algorithms, dynamic memory allocation, and input/output operations.
  - Helped in understanding and applying best practices for writing efficient and maintainable C code.
  - <https://www.geeksforgeeks.org/>

### TutorialsPoint

- [TutorialsPoint - C

Programming](<https://www.tutorialspoint.com/cprogramming/index.htm>)

- Provides detailed tutorials, reference guides, and examples covering basic to advanced C programming topics.

**Cprogramming.com**

- [Cprogramming.com](https://www.cprogramming.com/)
- A resource-rich site offering tutorials, articles, and tips for C and C++ programming.

**Learn-C.org**

- [Learn-C.org](https://www.learn-c.org/)
- An interactive tutorial platform for learning C programming through hands-on exercises and examples.

**W3Schools**

- [W3Schools - C Tutorial](https://www.w3schools.in/c-tutorial/)
- Offers a well-structured tutorial covering the basics of C programming along with examples and exercises.

**C Plus Plus (cplusplus.com)**

- [C Plus Plus - C Library Reference](http://www.cplusplus.com/reference/clibrary/)
- Provides a reference guide for the C standard library functions, useful for understanding and using library functions effectively.

**Stack Overflow**

- [Stack Overflow - C Programming Questions](https://stackoverflow.com/questions/tagged/c)
- A popular community-driven platform where you can ask questions and find answers related to C programming.